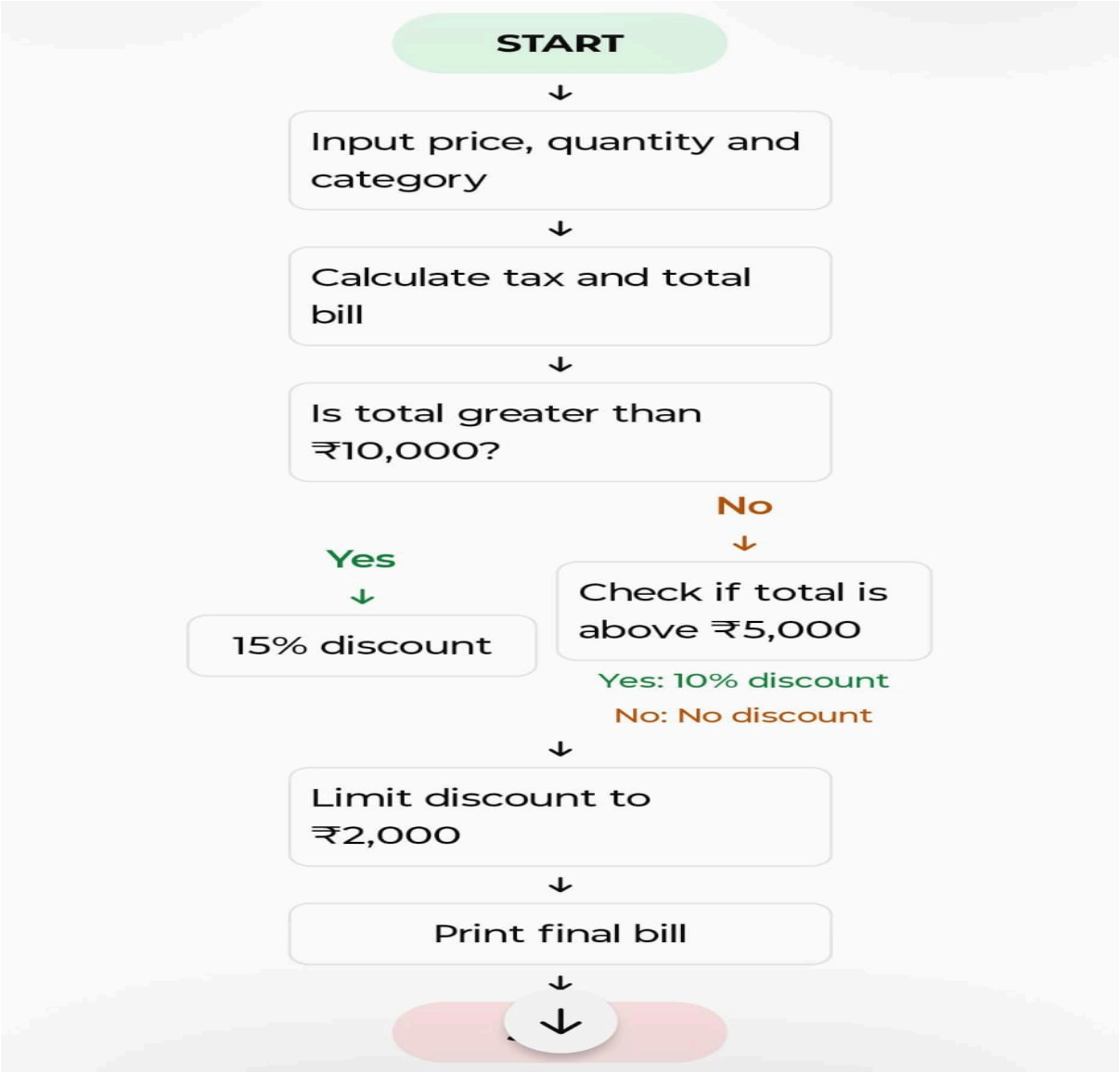
CASE 1: Retail Billing System

Objective : To calculate the bill using category-wise tax and discounts

Algorithm :

1. Start.
2. Input price, quantity, and category.
3. Calculate tax based on category.
4. Calculate total bill.
5. Apply discount based on the total.
6. Display the final bill.
7. Stop.

Flowchart :



Code :

```
a = input("Team 1: ")
b = input("Team 2: ")

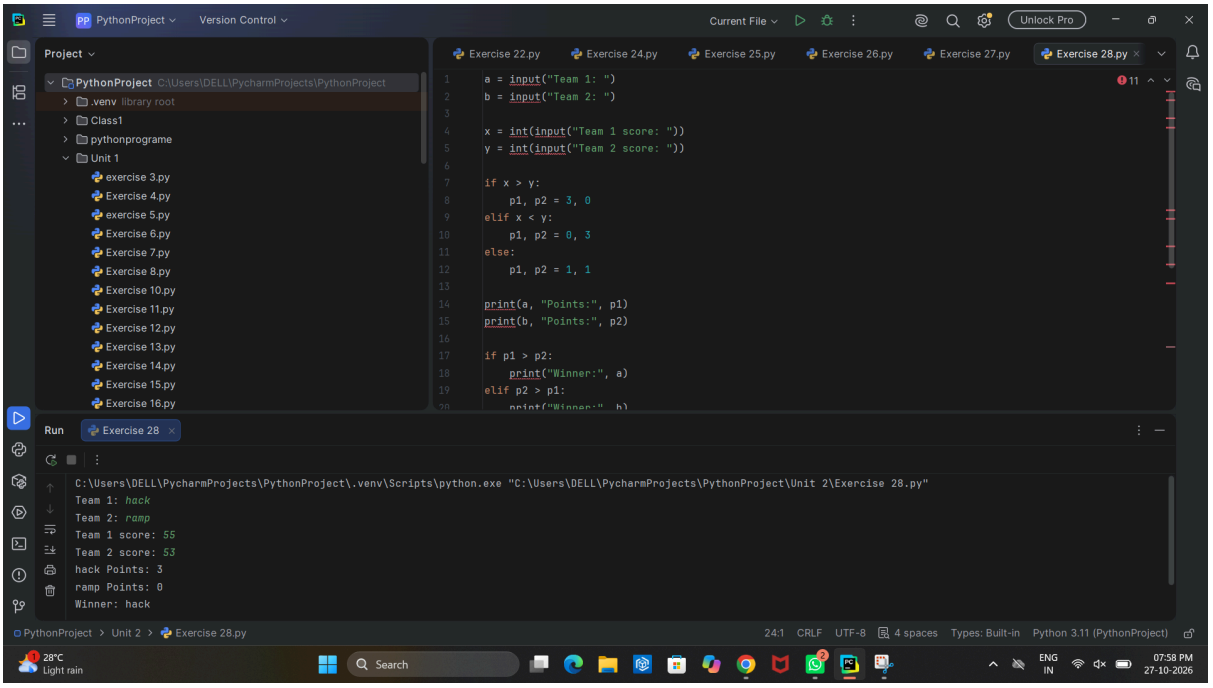
x = int(input("Team 1 score: "))
y = int(input("Team 2 score: "))

if x > y:
    p1, p2 = 3, 0
elif x < y:
    p1, p2 = 0, 3
else:
    p1, p2 = 1, 1

print(a, "Points:", p1)
print(b, "Points:", p2)

if p1 > p2:
    print("Winner:", a)
elif p2 > p1:
    print("Winner:", b)
else:
    print("Match Draw")
```

Output :



Conclusion :

This program calculates points and identifies the winner using simple if-elif-else statements.

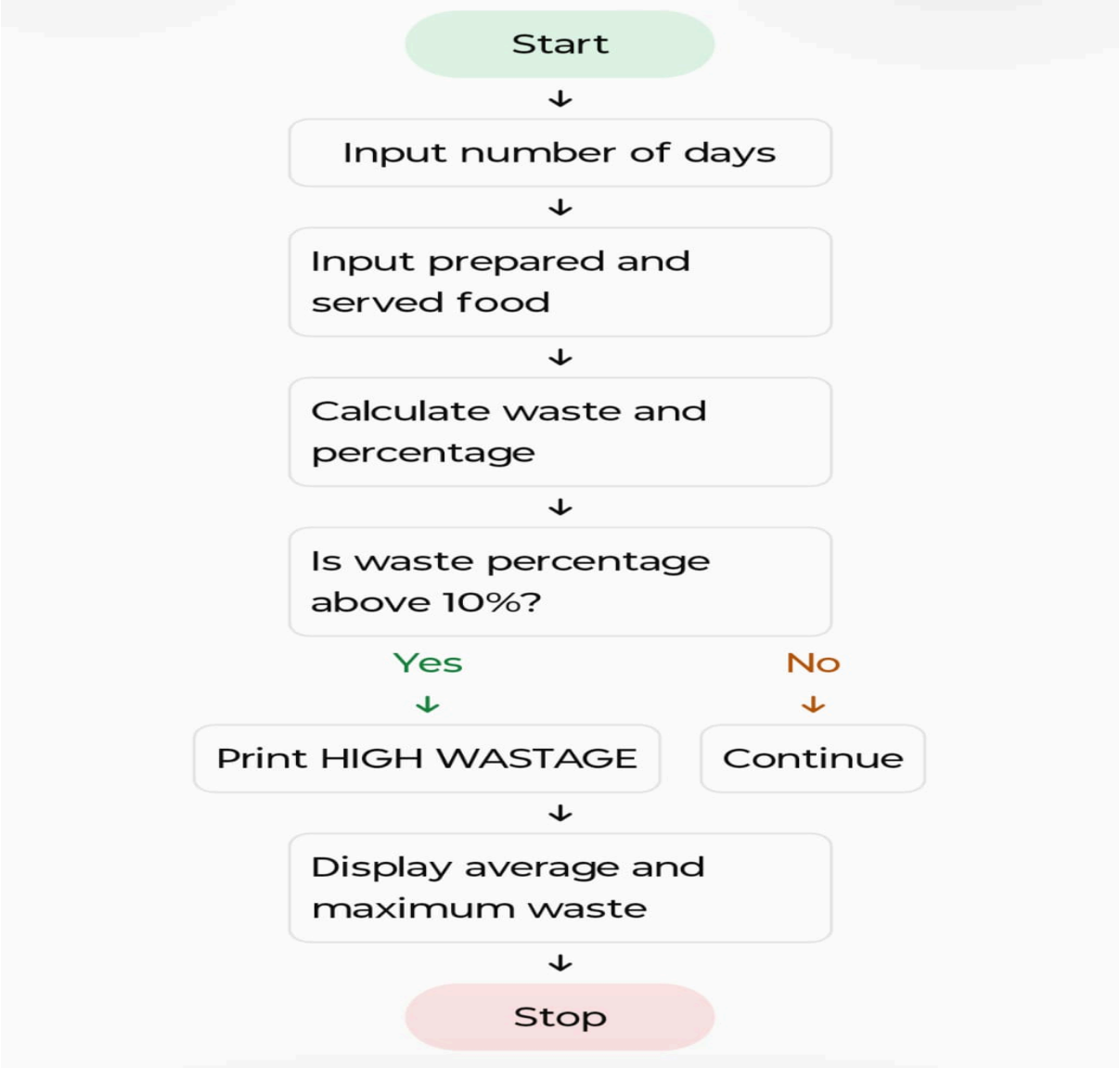
CASE 2: Food Waste Monitoring System

Objective : To track food wastage and flag high-wastage days to minimize days to minimize waste

Algorithm :

1. Start.
2. Input number of days.
3. Enter prepared and served food.
4. Calculate waste and waste percentage.
5. If waste is above 10%, display HIGH WASTAGE.
6. Calculate average waste percentage and maximum waste.
7. Stop.

Flowchart :



Code :

```
        total = 0
max_waste = 0

for i in range(1, 12):
    prepared = float(input("Food prepared: "))
    served = float(input("Food served: "))

    waste = prepared - served
    percent = (waste / prepared) * 100
    total += percent

    print("Wastage:", waste)
    print("Wastage %:", percent)

    if percent > 10:
        print("HIGH WASTAGE")

    if percent > max_waste:
        max_waste = percent
        max_day = i

print("Average wastage %:", total / 11)
print("Maximum wastage day:", max_day)
```

Output :

```
Food prepared: 7
Food served: 8
Wastage: -1.0
Wastage %: -14.285714285714285
```

Conclusion :

The program calculates food waste,finds the average and maximum waste,and identifies high-wastage days using python.

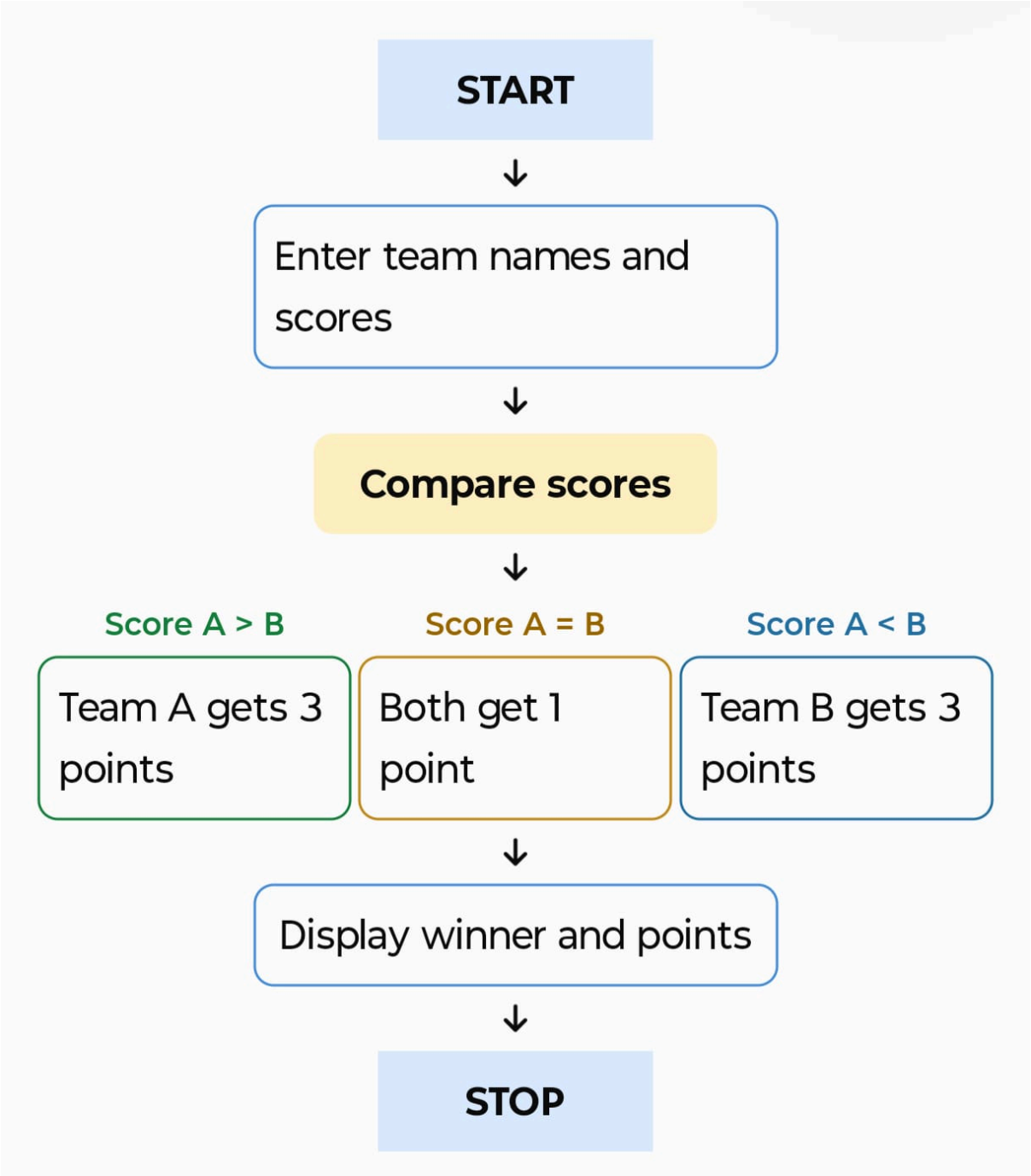
CASE 3 : Sports Tournament Ranking System

Objective : To compute points,apply tie-breakers,and generate tournament rankings.

Algorithm :

Start.
Read team names and scores.
Compare scores.
Print the result.
Stop.

Flowchart :



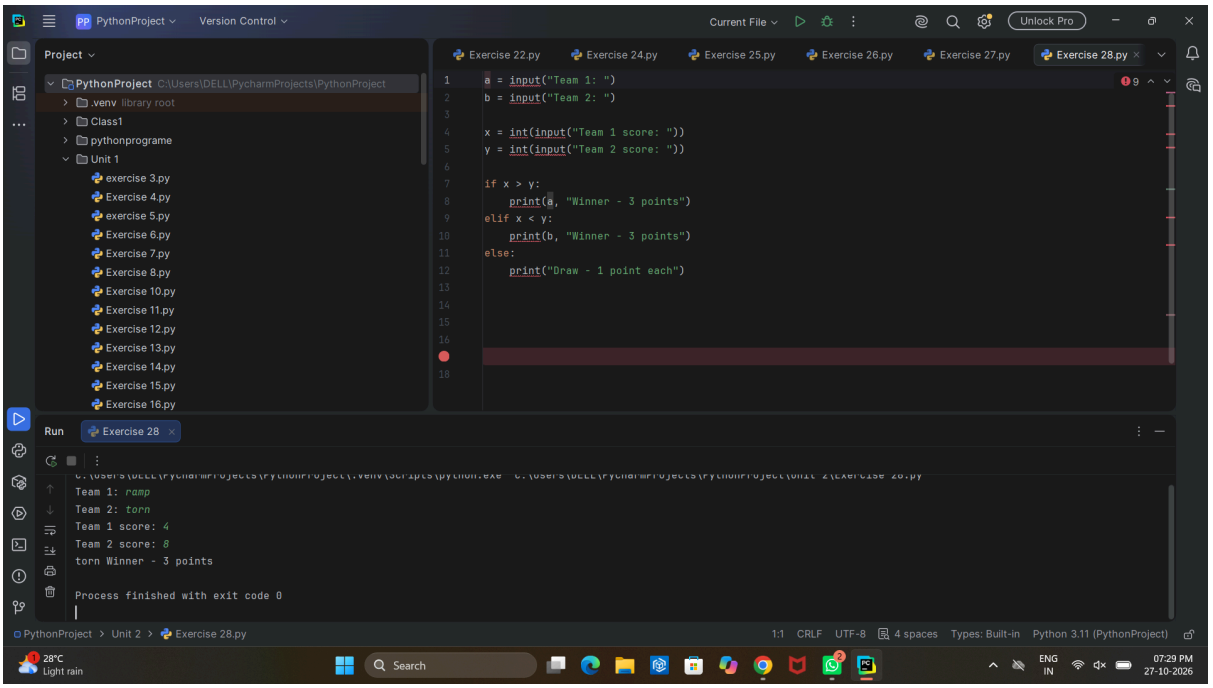
CODE :

```
 a = input("Team 1: ")
b = input("Team 2: ")

x = int(input("Team 1 score: "))
y = int(input("Team 2 score: "))

if x > y:
    print(a, "Winner - 3 points")
elif x < y:
    print(b, "Winner - 3 points")
else:
    print("Draw - 1 point each")
```

OUTPUT :



Conclusion :

The Program finds the match winner and awards points based on the scores.

CASE 4: Hackathon Team formation and Ranking

Objective : To check team size and calculate the total hackathon score.

Algorithm :

1.Start.
2.Read the number of teams.
3.Read team name and size.
4.If size is invalid, skip the team.
5.Add both round scores.
6.Display the total score.
7.Stop.

Flowchart :



Code :

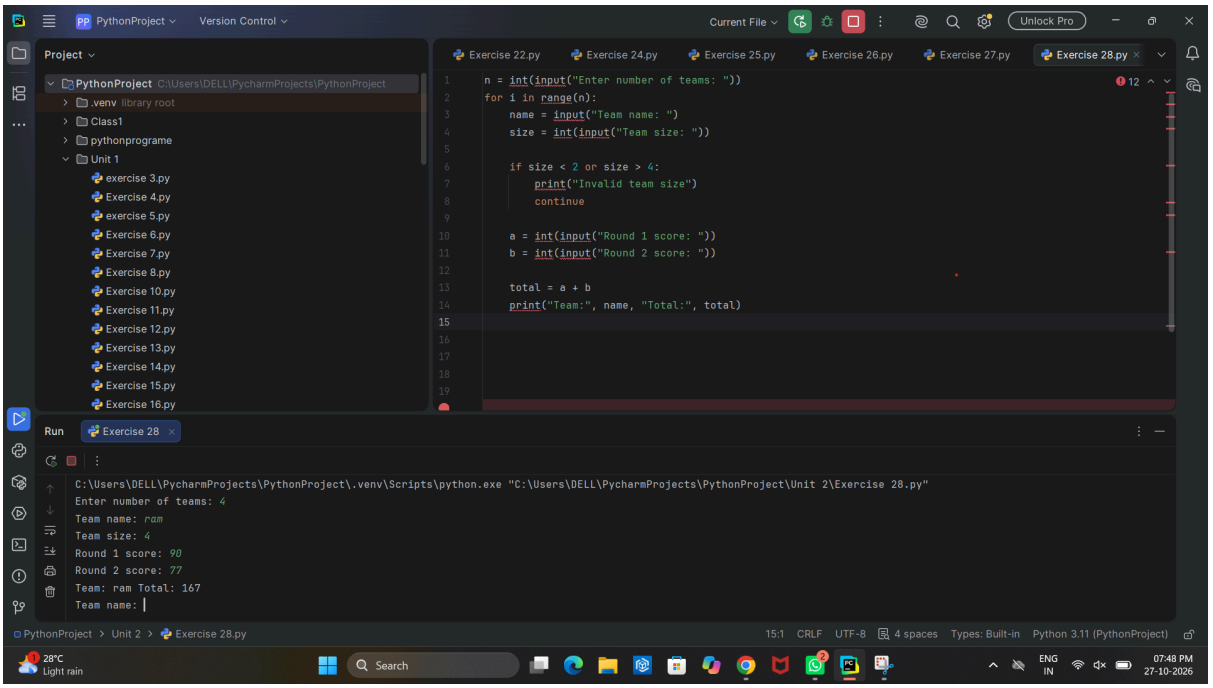
```
n = int(input("Enter number of teams: "))
for i in range(n):
    name = input("Team name: ")
    size = int(input("Team size: "))

    if size < 2 or size > 4:
        print("Invalid team size")
        continue

    a = int(input("Round 1 score: "))
    b = int(input("Round 2 score: "))

    total = a + b
    print("Team:", name, "Total:", total)
```

Output :



Conclusion :

Thus, the program checks team size and calculates the total score of each valid team.

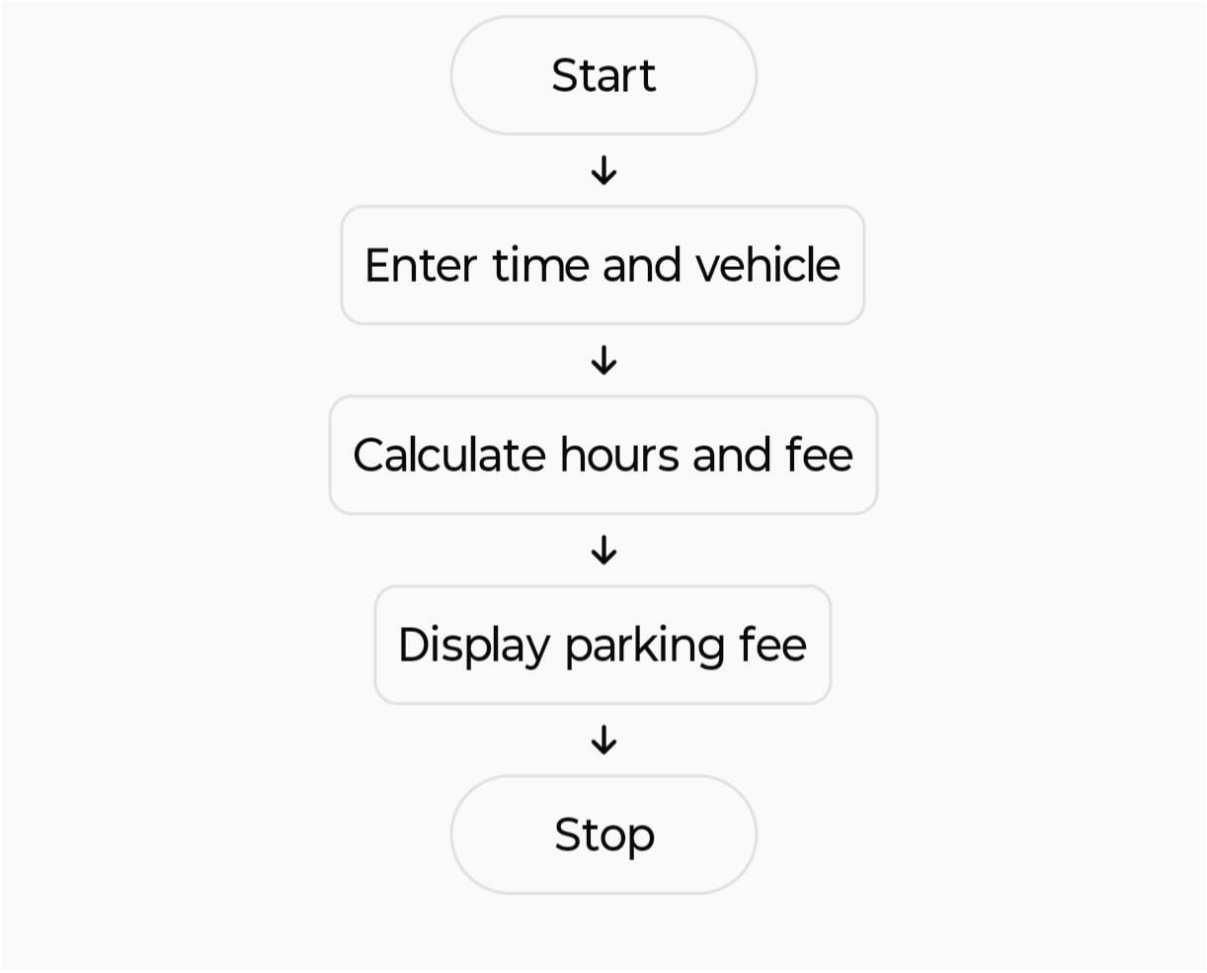
CASE 5 : Smart Packing Free Mnagement

Objective : To calculate parking fees based on vehicle type and parking hours.

Algorithm :

Start.
Read entry, exit and vehicle.
Validate time.
Calculate hours and fee.
Display parking fee.
Stop.

Flowchart :



Code :

```
 entry = int(input("Enter entry hour: "))
exit = int(input("Enter exit hour: "))
v = input("Enter vehicle (car/bike): ")

if entry < 0 or entry > 23 or exit < 0 or exit > 23:
    print("Invalid time")
else:
    hours = exit - entry

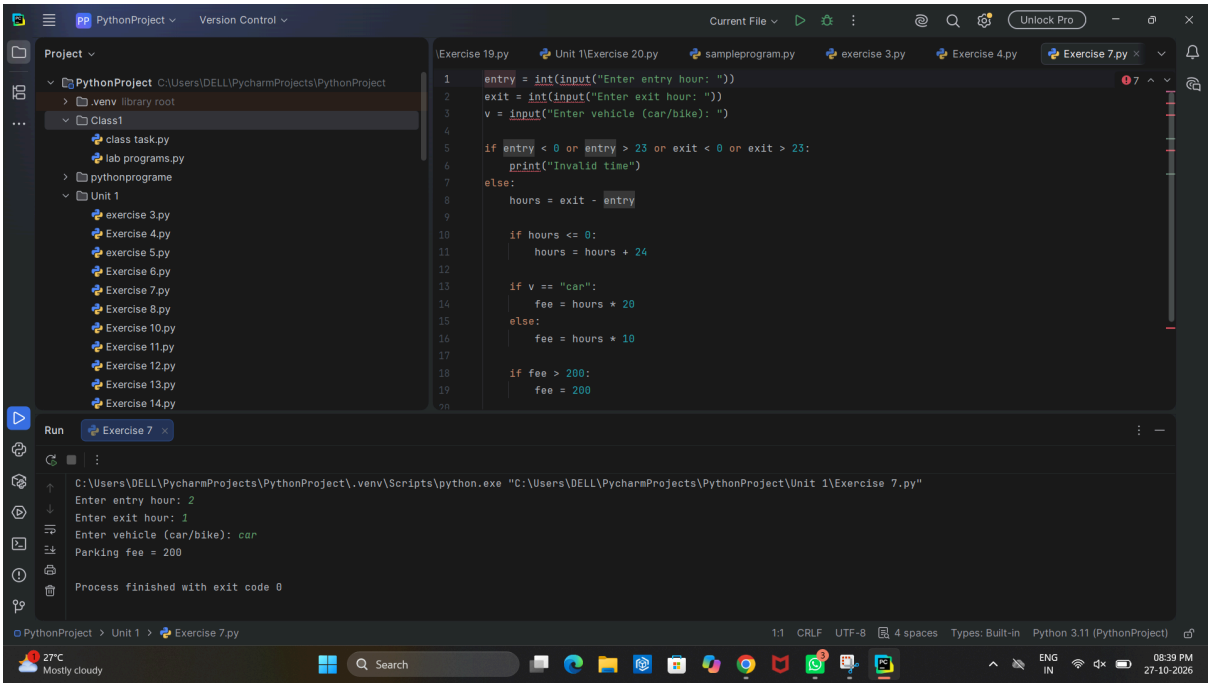
    if hours <= 0:
        hours = hours + 24

    if v == "car":
        fee = hours * 20
    else:
        fee = hours * 10

    if fee > 200:
        fee = 200

    print("Parking fee =", fee)
```

Output :



Conclusion :

Thus the program calculates the parking fee based on vehicle type and parking duration.

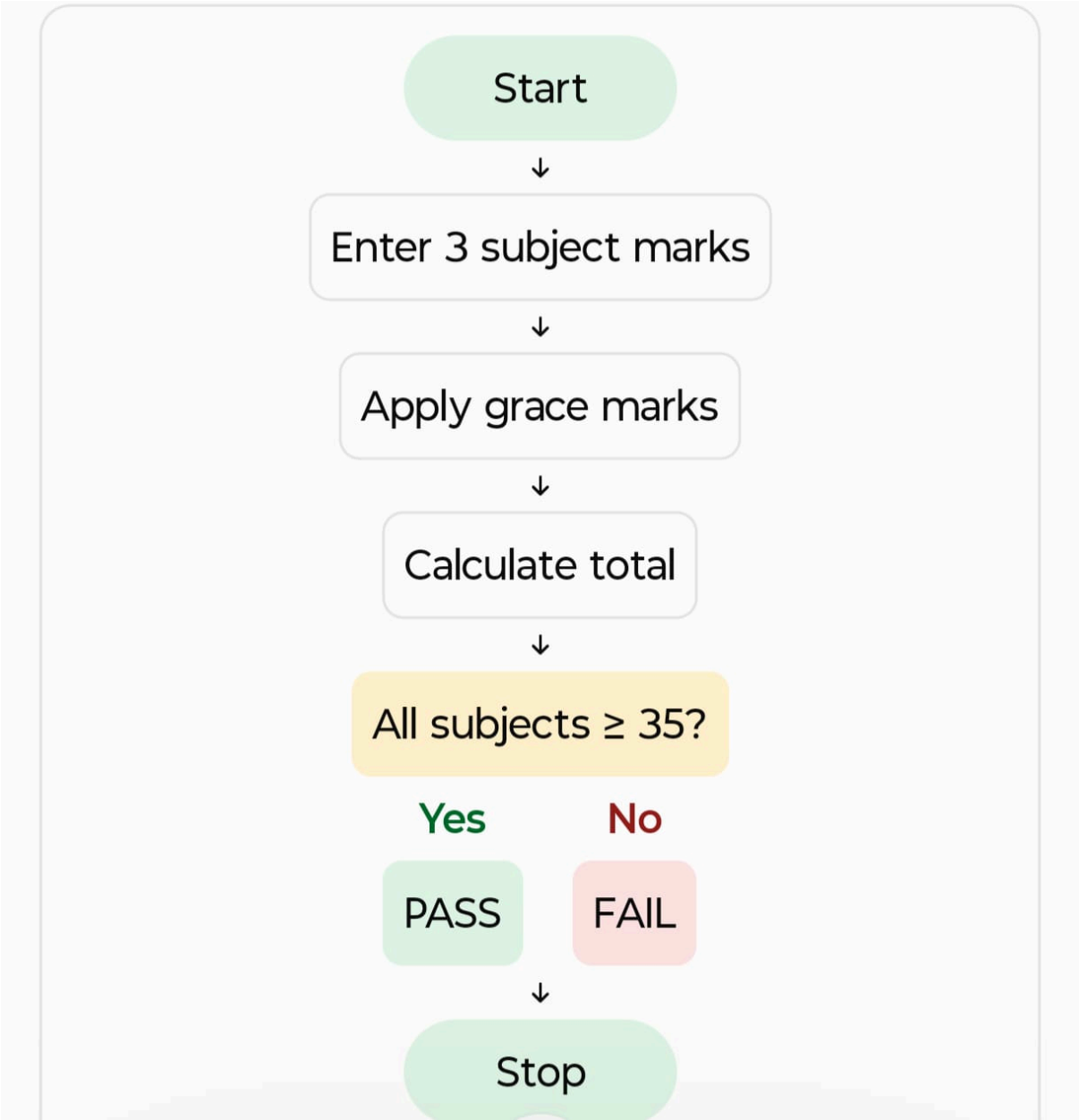
CASE 6 : Examination Result and Grace-Mark Validation

Objective : To calculate total marks and display PASS or FAIL using grace marks

Algorithm :

1. Start.
2. Read three subject marks.
3. Add 5 grace marks if marks are between 30 and 34.
4. Calculate total marks.
5. Check pass criteria.
6. Print result.
7. Stop.

Flowchart :



Code :

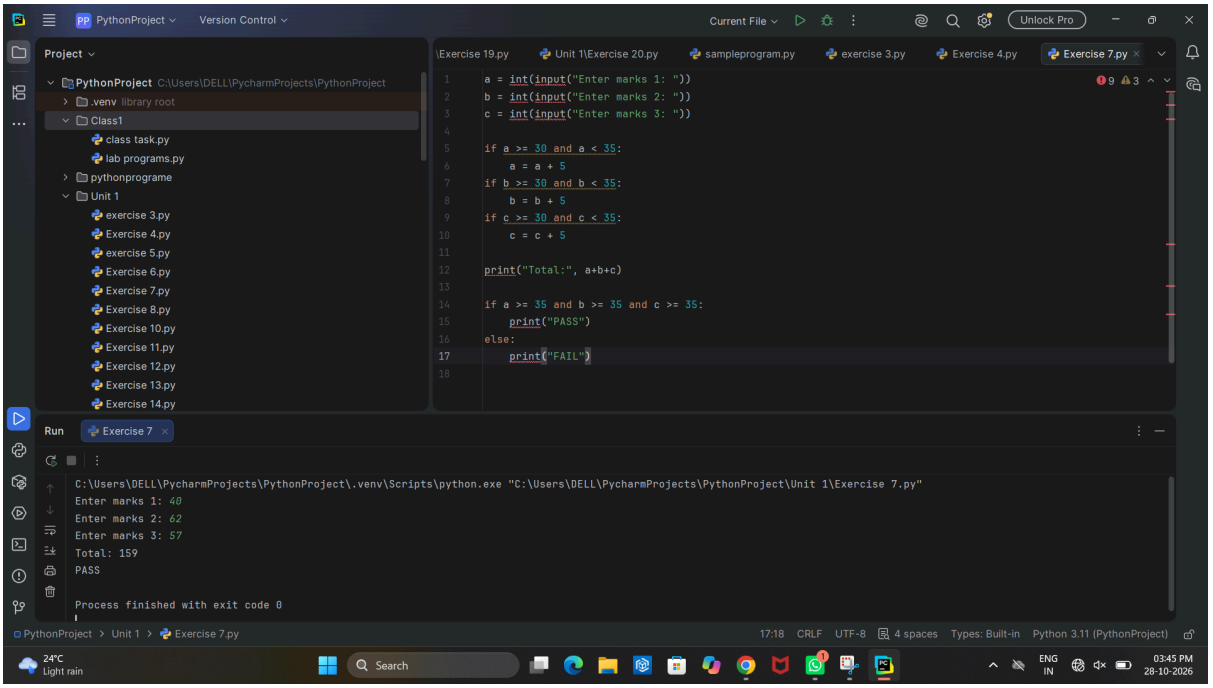
```
a = int(input("Enter marks 1: "))
b = int(input("Enter marks 2: "))
c = int(input("Enter marks 3: "))

if a >= 30 and a < 35:
    a = a + 5
if b >= 30 and b < 35:
    b = b + 5
if c >= 30 and c < 35:
    c = c + 5

print("Total:", a+b+c)

if a >= 35 and b >= 35 and c >= 35:
    print("PASS")
else:
    print("FAIL")
```

Output :



Conclusion :

Thus, the program calculates total marks and displays the final result using grace marks.

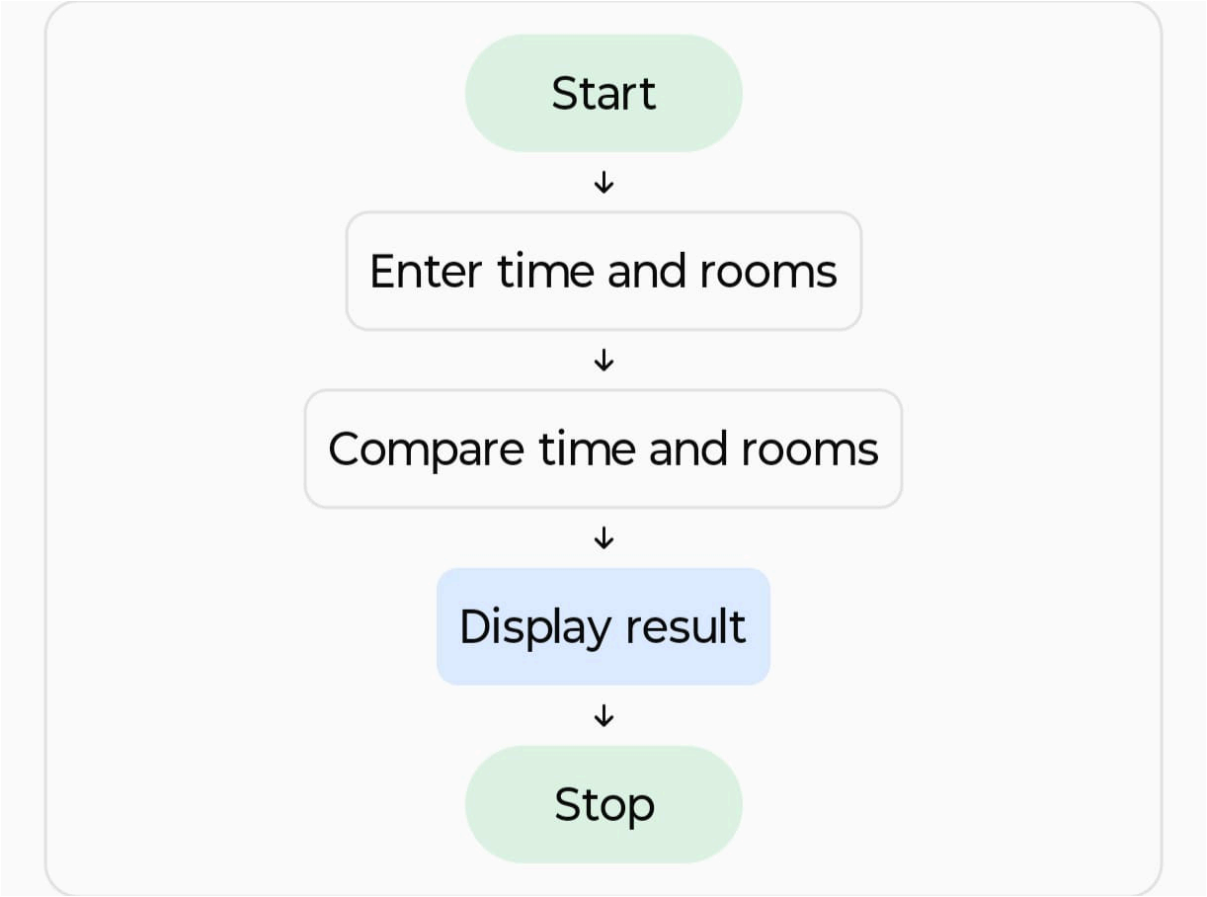
CASE 7 : Student Timetable Conflict Detection

Objective : To Check timetable and room conflicts.

Algorithm :

1.Start.
2.Read timings and rooms.
3.Compare timings.
4.Compare rooms.
5.Print the result.
6.Stop.

Flowchart :



Code :

```
a = input("Enter time 1: ")
b = input("Enter time 2: ")

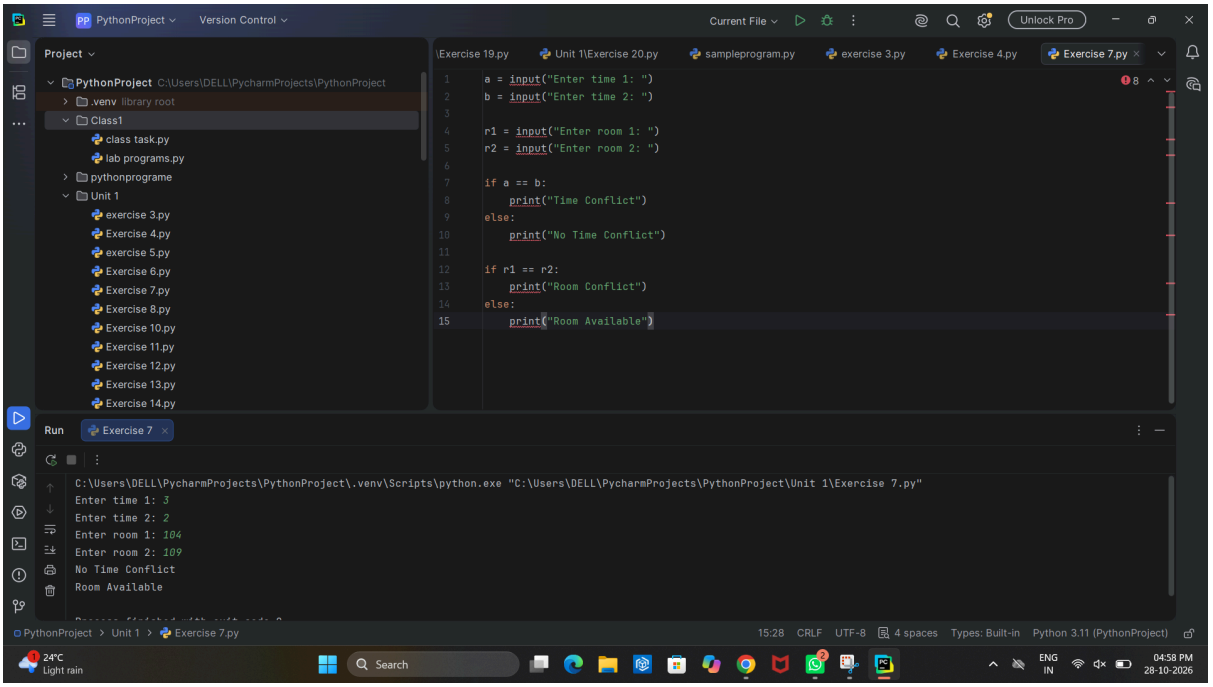
r1 = input("Enter room 1: ")
r2 = input("Enter room 2: ")

if a == b:
    print("Time Conflict")
else:
```

```
    print("No Time Conflict")

if r1 == r2:
    print("Room Conflict")
else:
    print("Room Available")
```

Output :



Conclusion :

Thus, the program checks and room conflicts using simple python conditions.

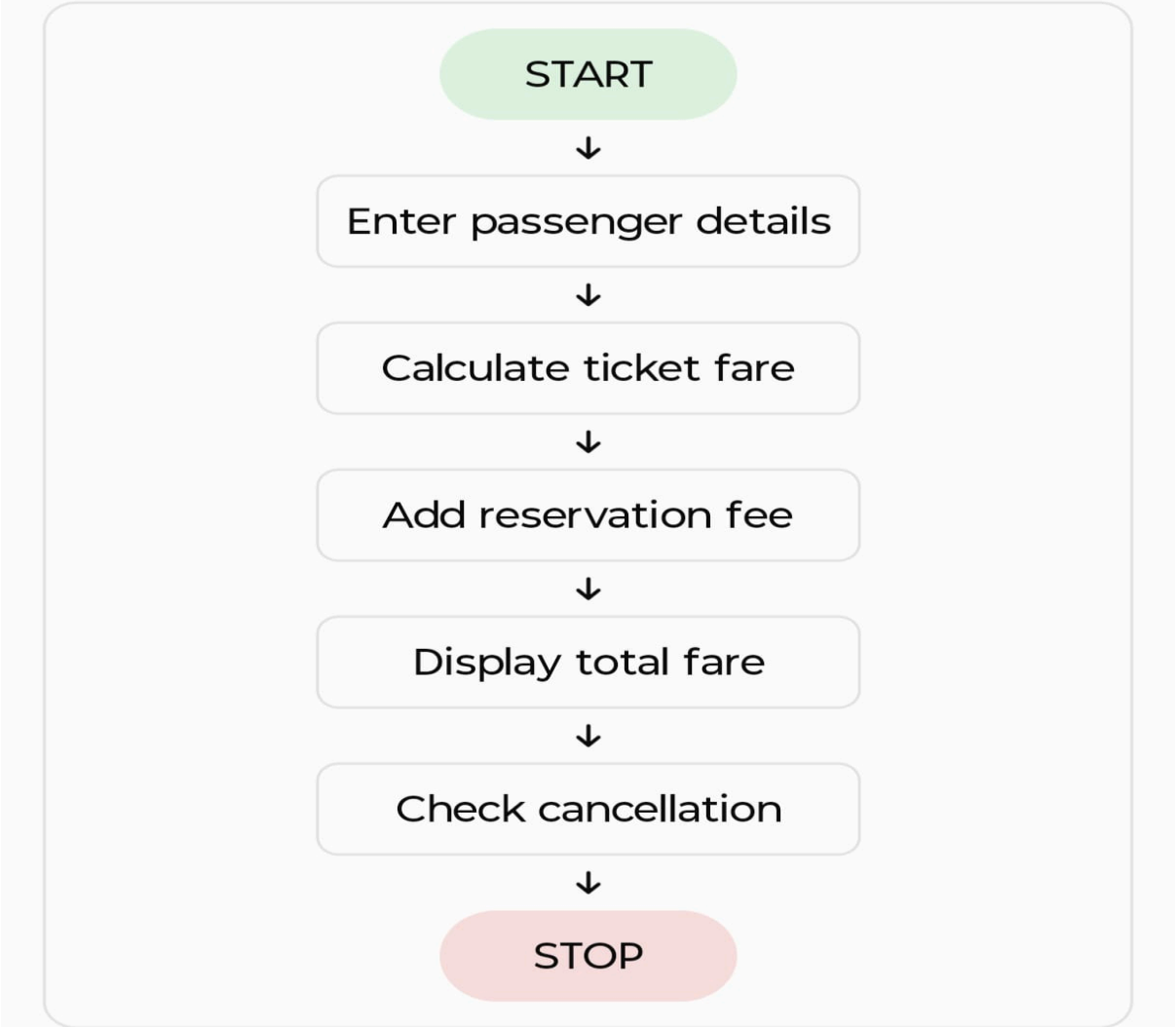
CASE 8 : Railway Fare and Reservation Calculator

Objective : To calculate railway ticket fare based on distance,class,and age.

Algorithm :

Start.
Read passenger details.
Calculate fare.
Apply concession and add ₹50 fee.
Display total fare.
Check cancellation and calculate refund.
Stop.

Flowchart :



Code :

```
n = int(input("Enter passengers: "))
total = 0

for i in range(n):
    d = int(input("Enter distance: "))
    age = int(input("Enter age: "))
    c = int(input("Enter class (1 or 2): "))

    fare = d

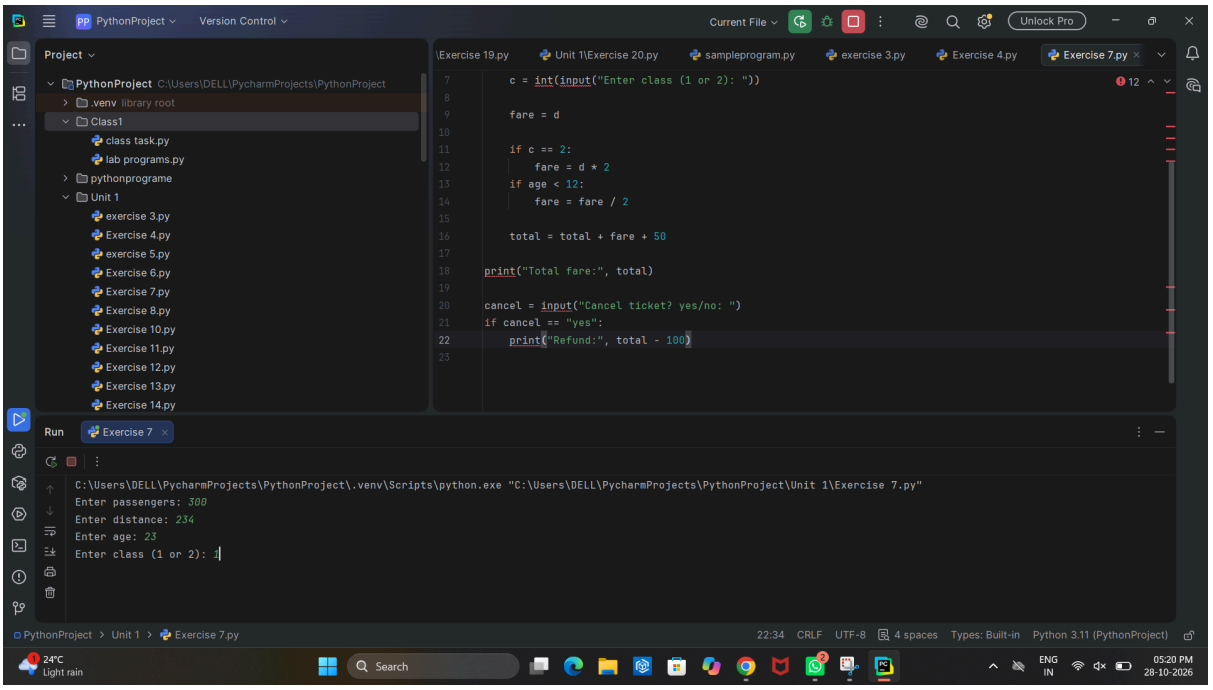
    if c == 2:
        fare = d * 2
    if age < 12:
        fare = fare / 2

    total = total + fare + 50

print("Total fare:", total)

cancel = input("Cancel ticket? yes/no: ")
if cancel == "yes":
    print("Refund:", total - 100)
```

Output :



Conclusion :

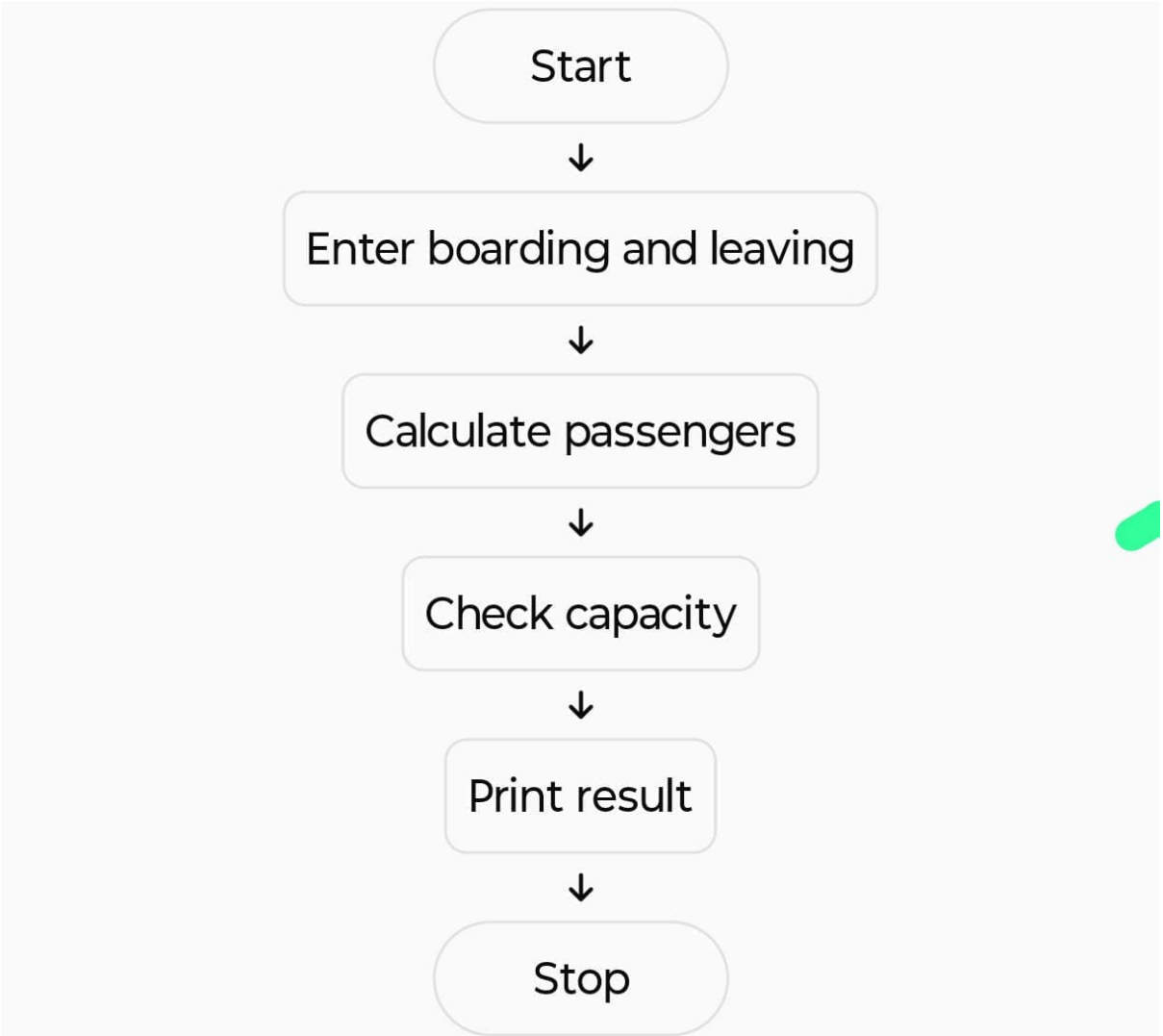
CASE 9 : Bus Passenger Occupancy Simulation

Objective : To count bus passenger and check the bus capacity.

Algorithm :

1.Start.
2.Set capacity = 50 and passengers = 0.
3.Enter boarding and leaving counts.
4.Calculate passengers.
5.Check capacity and print the result.
6.Repeat for 3 stops.
7.Stop.

Flowchart :



Code :

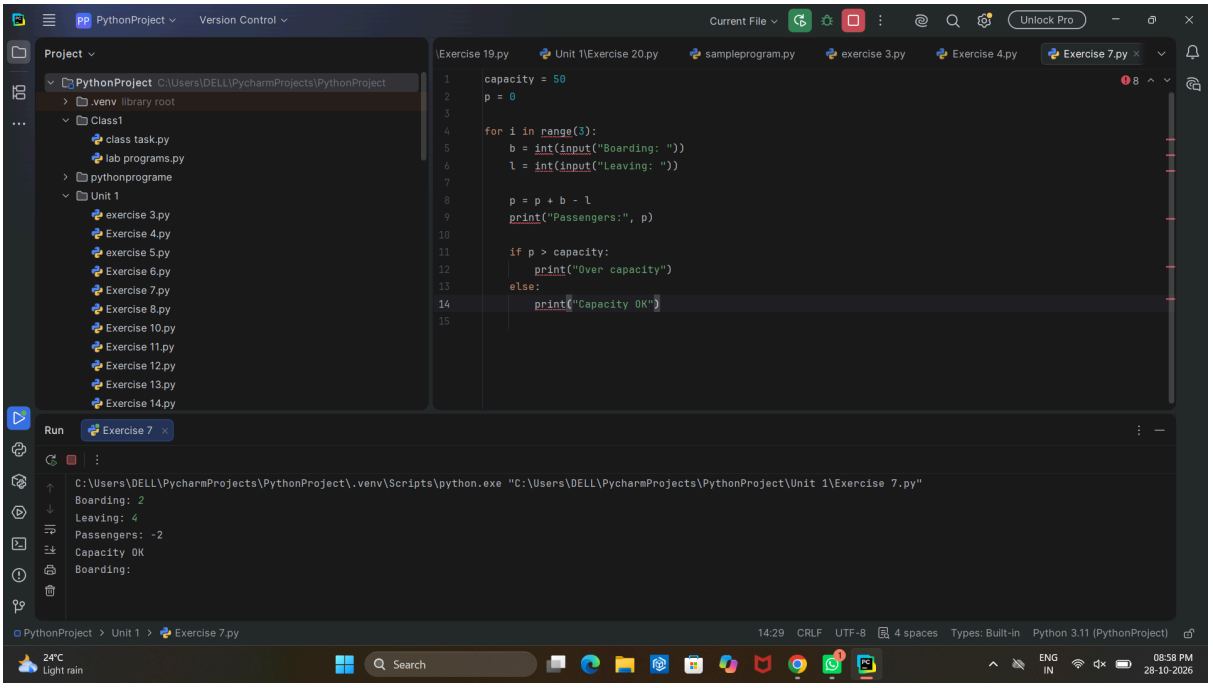
```
capacity = 50
p = 0

for i in range(3):
    b = int(input("Boarding: "))
    l = int(input("Leaving: "))

    p = p + b - l
    print("Passengers:", p)

    if p > capacity:
        print("Over capacity")
    else:
        print("Capacity OK")
```

Output :



Conclusion :

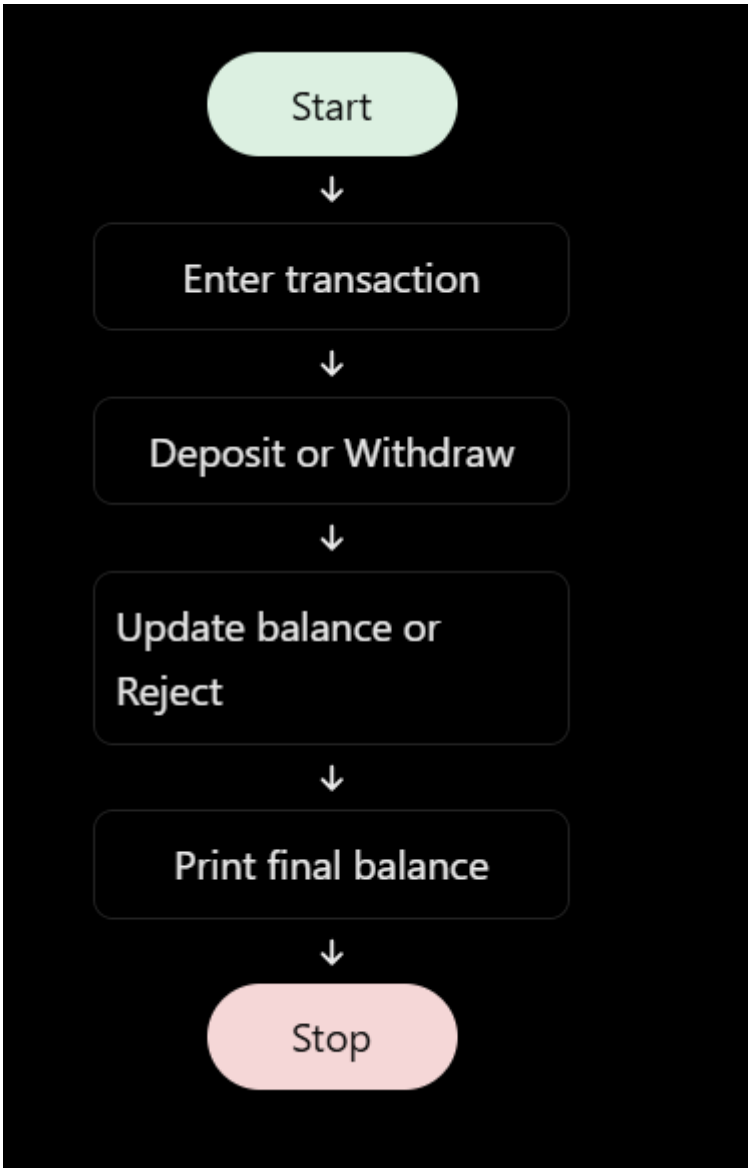
The program counts passengers and checks whether the bus exceeds its capacity.

CASE 9 : ATM Transaction Simulation

Objective : To Perform deposits and withdrawals and display the final balance.

Algorithm :
1.Start.
2.Set balance to 1000.
3.Read transactions.
4.Deposit or withdraw money.
5.Reject invalid transactions.
6.Print final balance.
7.Stop.

Flowchart :



Code :

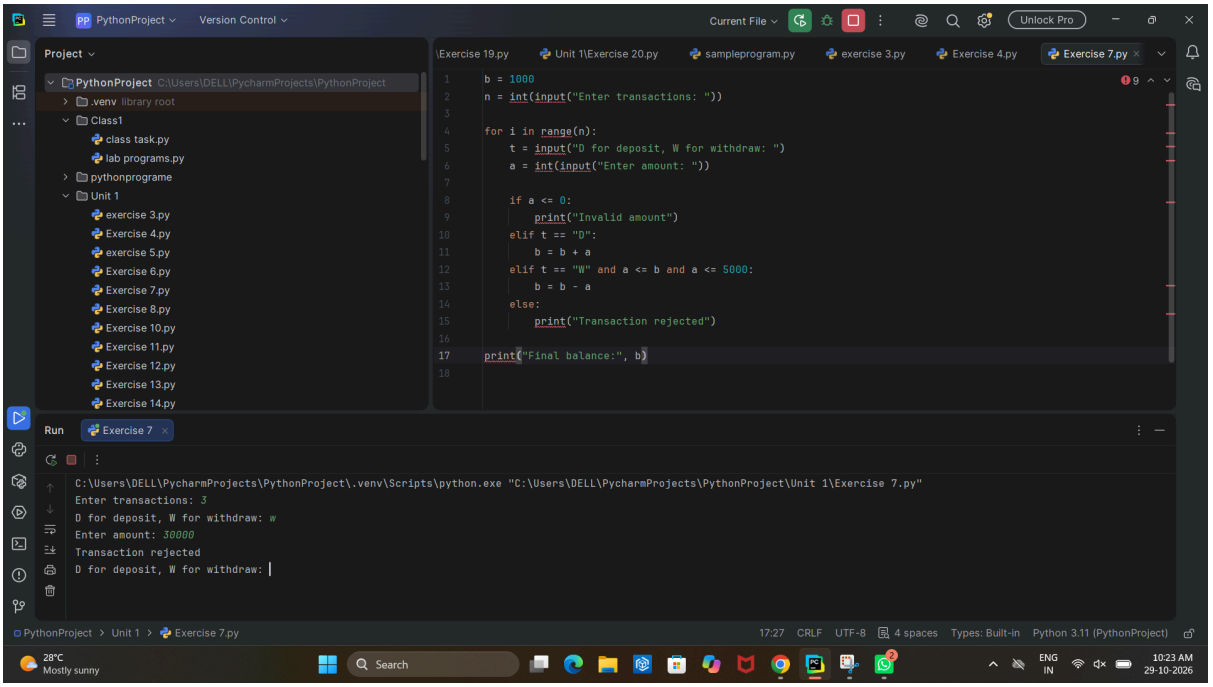
```
b = 1000
n = int(input("Enter transactions: "))

for i in range(n):
    t = input("D for deposit, W for withdraw: ")
    a = int(input("Enter amount: "))

    if a <= 0:
        print("Invalid amount")
    elif t == "D":
        b = b + a
    elif t == "W" and a <= b and a <= 5000:
        b = b - a
    else:
        print("Transaction rejected")

print("Final balance:", b)
```

Output :



Conclusion :

The program performs ATM transactions and displays the final balance.

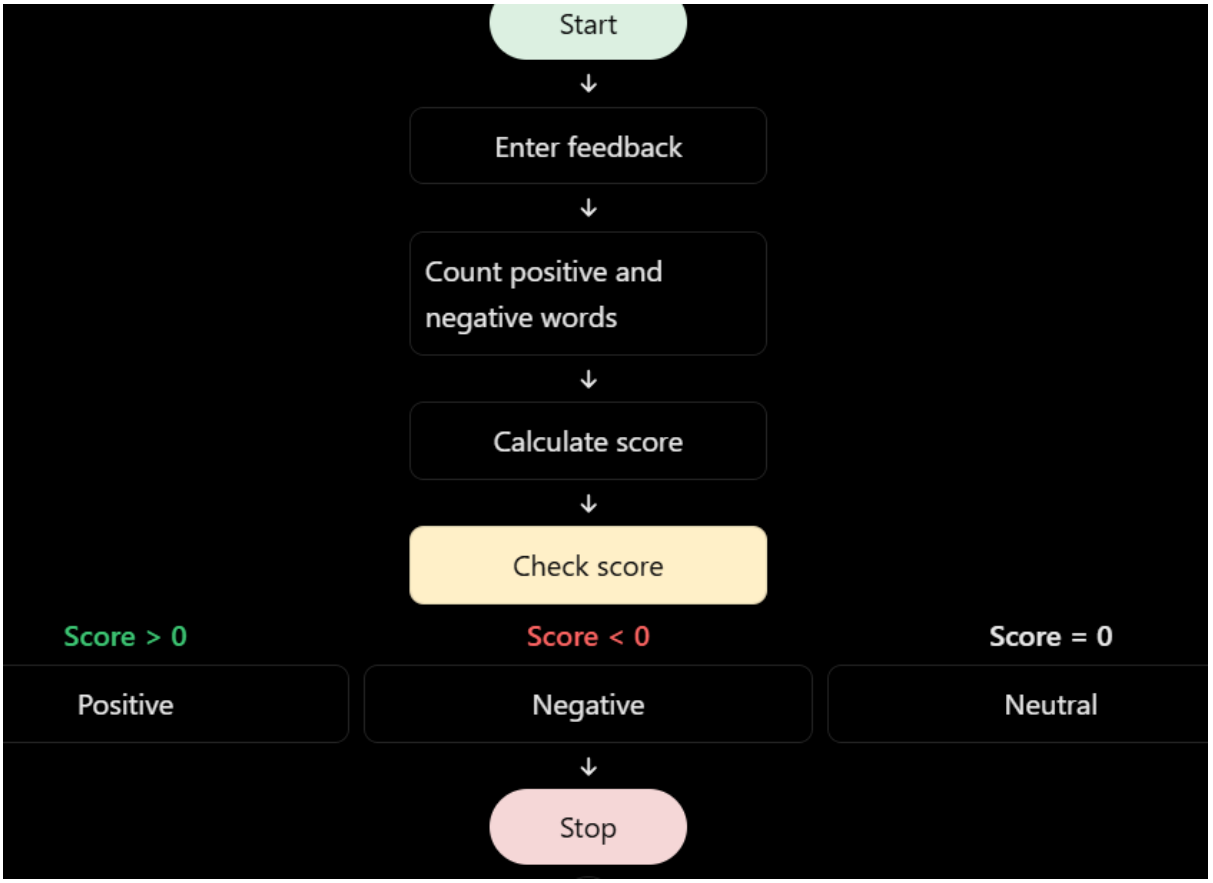
CASE 10 : Student feedback sentiment Scoring

Objective : To analyse student feedback and classify it as Positive, Negative, or Neutral.

Algorithm :

1. Start.
2. Read feedback.
3. Count positive keywords as +1 and negative keywords as -1.
4. Calculate the score.
5. Classify the feedback.
6. Stop.

Flowchart :



Code :

```
p = ["good", "excellent", "happy"]
n = ["bad", "poor", "worst"]

f = input("Enter feedback: ").lower()
score = 0

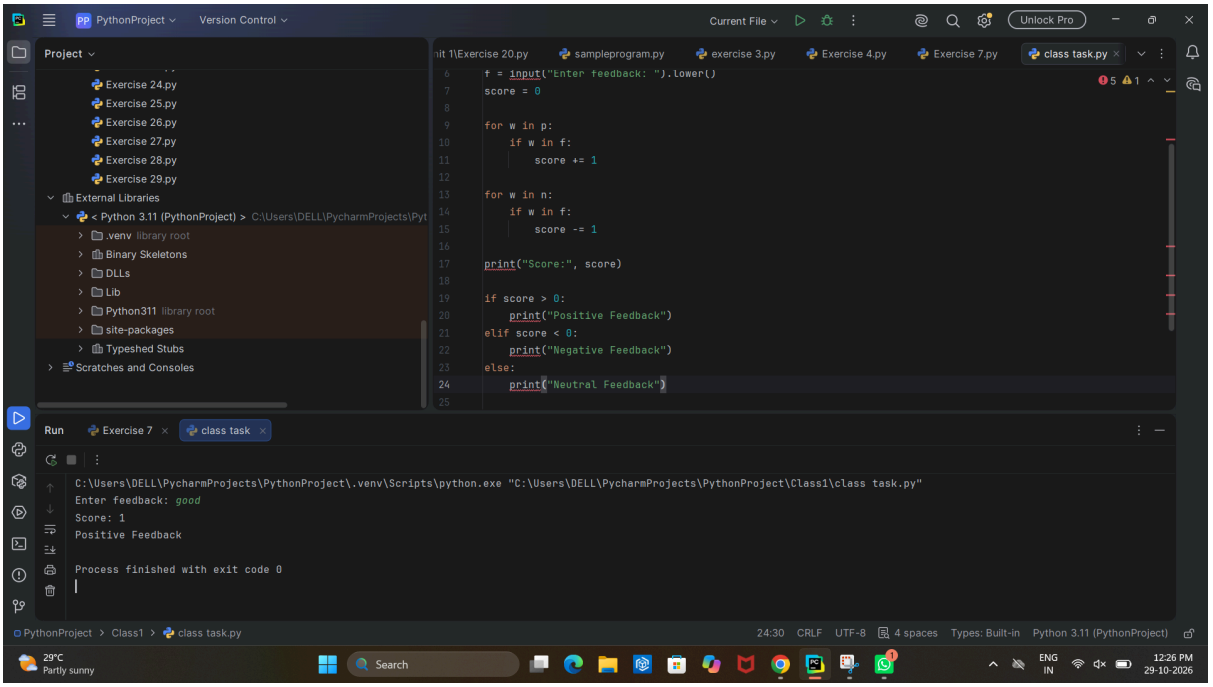
for w in p:
    if w in f:
        score += 1
```

```
for w in n:
    if w in f:
        score -= 1

print("Score:", score)

if score > 0:
    print("Positive Feedback")
elif score < 0:
    print("Negative Feedback")
else:
    print("Neutral Feedback")
```

Output :



Conclusion :

The program analyses feedback keywords, calculates a score, and classifies feedback as positive, negative, or neutral.